

R기초와 데이터 마트

• MODULE 6

모듈 6-1

R 기초

• R 기초

- 분석 환경의 이해

- R은 뉴질랜드 통계학자인 로스 이하카(Ross Ihaka)와 캐나다 통계학자인 로버트 젠틀맨(Robert Gentleman)에 의해 1995년 제작된 언어로 다양한 분야에서 사용되는 **오픈소스 통계 분석 도구**임
- R의 특징은 다음과 같음

특징	설명
오픈소스	• 오픈소스 소프트웨어이므로 자유로운 사용이 가능
인터프리터 언어	• 프로그램 문장을 하나씩 번역하고 실행하는 언어
대소문자 구별	• 식별자, 명령어 등 대소문자 구분
범용성	• Microsoft Windows, Mac OS, Linux 등 다양한 OS(운영체제) 지원
다양한 기능	• 다양한 최신 통계 분석과 데이터마이닝 기능 제공

- R 기초

- 분석 환경의 이해

- R 설치 방법

- ✓ <http://r-project.org>에서 무료로 다운받을 수 있음
 - ✓ 좌측 Download 메뉴의 CRAN을 클릭함
 - ✓ Korea 서버를 찾아 클릭하여 운영체제에 맞게 다운로드 링크를 선택함
 - ✓ R을 처음 설치한다면 R base만 설치하면 됨

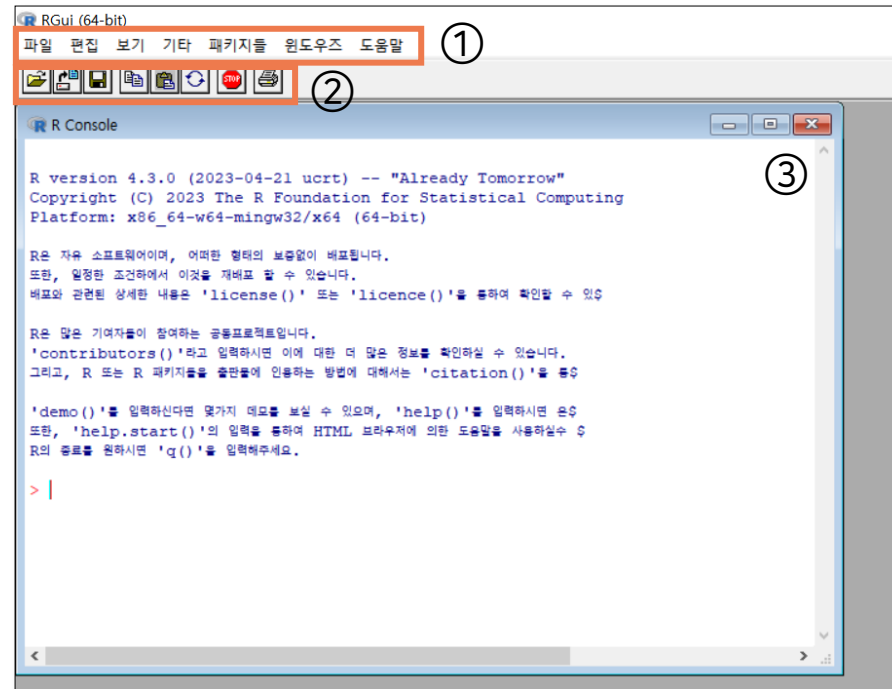
R 기초와 데이터 마트

• R 기초

- 기본 사용법

➤ R의 구성

- ✓ ① 메뉴 바
- ✓ ② 단축아이콘 톨바
- ✓ ③ R 콘솔: R 명령어가 입력되는 공간



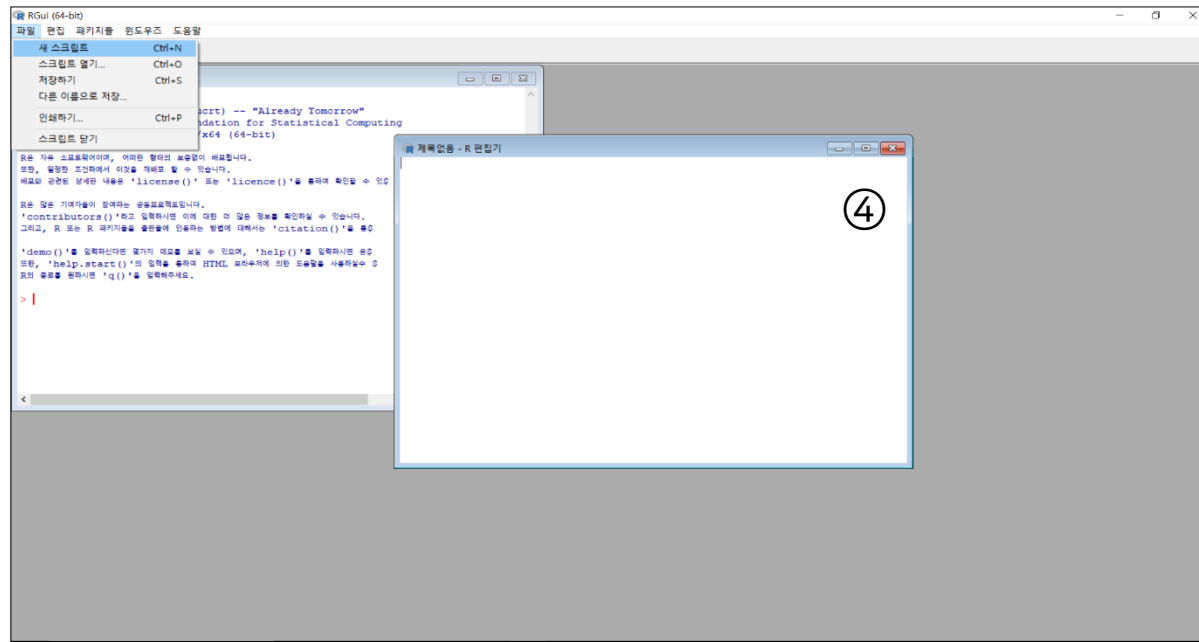
R 기초와 데이터 마트

- R 기초

- 기본 사용법

- R의 구성

✓ ④ 스크립트 창: 여러 줄의 코드를 한번에 작성할 때 사용됨



- R 기초

- 기본 사용법

- 변수

- ✓ 변수는 다양한 값을 지니고 있는 하나의 속성
 - ✓ 변수명은 문자, 숫자, 언더바(_), 마침표(.)를 조합해서 생성함
 - ✓ 변수명의 첫 글자는 마침표 또는 문자로 시작해야 하며 마침표로 시작할 경우 뒤에 숫자가 올 수 없음
 - ✓ 변수명은 대소문자를 구분함
 - ✓ 올바른 변수명: a, bc_a, a_1, a
 - ✓ 잘못된 변수명: 3x, 1

• R 기초

- 기본 사용법

➤ 데이터 타입

- ✓ 데이터 타입은 프로그래밍 언어에서 실수형, 문자형과 같은 여러 종류의 데이터를 식별하는 형태임
- ✓ 데이터 기본 타입으로 숫자형, 문자형, 논리형이 있음

유형	설명
숫자형	• 정수, 실수를 저장하는 객체 형식
문자형	• 문자, 문자열을 저장하는 객체 형식
논리형	• 논리값 TRUE, FALSE를 저장하는 객체 형식

- ✓ 객체: 메모리에 생성된 것

• R 기초

- 기본 사용법

➤ 데이터 타입

✓ 데이터의 특수한 값으로 NA, NULL, NaN, INF가 있음

값	설명
NA(Not Available)	- 데이터의 값이 없음(결측값) - 공간을 차지하는 결측값
NULL	- 변수가 초기화되지 않는 경우 - 공간을 차지하지 않는 결측값
NaN(Not a Number)	수학적으로 계산할 수 없는 값
INF(INFinite)	무한대

• R 기초

- 기본 사용법

➤ 대입 연산자

- ✓ 변수를 선언하고 값을 할당할 때는 대입 연산자를 사용함

대입 연산자	설명
=	• 왼쪽의 변수에 오른쪽의 값을 대입하는 연산자
<-, <<-	• 왼쪽의 변수에 오른쪽의 값을 대입하는 연산자
->, ->>	• 오른쪽의 변수에 왼쪽의 값을 대입하는 연산자

- ✓ a = "big" (a 변수에 big이라는 문자열 할당)
- ✓ b <- 3 (b 변수에 숫자 3을 할당)

• R 기초

- 기본 사용법

➤ 비교 연산자

✓ 대입 연산자에 의해 할당된 값과 변수, 숫자, 문자 및 논리값을 비교할 수 있음

비교 연산자	설명
==	• 두 값이 같은지 비교함
<, >	• 두 값의 초과와 미만을 비교함
!=	• 두 값이 다른지를 비교함
<=, >=	• 두 값의 이상과 이하를 비교함
is.character	• 문자형인지 비교함
is.numeric	• 숫자형인지 비교함
is.logical	• 논리형인지 비교함
is.na	• NA인지 비교함

• R 기초

- 기본 사용법

➤ 산술 연산자

✓ 두 숫자형 타입의 계산을 위한 연산자로 다양한 연산이 가능함

산술 연산자	설명
+	• 두 숫자의 덧셈
-	• 두 숫자의 뺄셈
*	• 두 숫자의 곱셈
/	• 두 숫자의 나눗셈
%%/%	• 두 숫자의 나눗셈의 몫
%%	• 두 숫자의 나눗셈의 나머지
^, **	• 거듭제곱

• R 기초

- 기본 사용법

➤ 기타 연산자

- ✓ 논리값을 계산하기 위한 연산자로 부정 연산자, AND 연산자, OR 연산자가 있음

기타 연산자	설명
!	• 현재의 논리값에 반대되는 값을 의미함
&	• 두 값이 모두 참일 때만 참이 됨
	• 둘 중 하나의 값만 참이더라도 참을 반환함

```
> !TRUE  
[1] FALSE  
> TRUE&FALSE  
[1] FALSE  
> TRUE|FALSE  
[1] TRUE  
> TRUE&TRUE  
[1] TRUE
```

- R 기초

- 기본 사용법

- 객체

- ✓ 객체의 종류로 벡터(Vector), 행렬(Matrix), 배열(Array)은 한 가지 타입을 가진 원소들만 저장할 수 있으며, 리스트(list), 데이터프레임(Data Frame)은 다양한 타입을 가진 원소들도 저장할 수 있음

차원	단일 자료	다중 자료
1차원	벡터	리스트
2차원	행렬	데이터 프레임
3차원	배열	

- R 기초

- 기본 사용법

- 벡터

- ✓ 벡터는 기본적인 데이터 단위로 같은 데이터 타입을 가진 원소들만 저장할 수 있는 타입임
 - ✓ 다른 타입의 데이터를 섞어 벡터에 저장하면 이들 데이터는 한 가지 타입으로 형태가 변환됨
 - ✓ 벡터 데이터 내에 들어갈 수 있는 원소는 숫자, 문자, 논리 연산자 등이 될 수 있음
 - ✓ 숫자로 이루어진 벡터는 숫자 벡터, 문자로 이루어진 벡터는 문자 벡터가 됨
 - ✓ R에서 다루는 데이터 구조 중 가장 단순한 형태는 명령어 c를 이용해 선언할 수 있음
 - ✓ c는 combine(결합)을 의미함

- R 기초

- 기본 사용법

- 벡터

- ✓ $x = c(1, 2, 3, 4)$

- ✓ $y = c(\text{"사과"}, \text{"배"}, \text{"바나나"})$

- ✓ $z = c(\text{TRUE}, \text{FALSE}, \text{TRUE})$

- ✓ x 는 숫자형 벡터, y 는 문자열 벡터, z 는 논리연산자 벡터가 됨

- ✓ 논리연산자 벡터를 숫자형 벡터처럼 사용하는 경우 TRUE는 1, FALSE는 0의 값을 할당받음

- R 기초

- 기본 사용법

- 벡터

- ✓ `x<-c(TRUE, 1, 2)`

- ✓ `y<-c(TRUE, "배")`

- ✓ `z<-c(TRUE, 1, "배")`

- ✓ 논리형 TRUE, 숫자 1, 2를 결합하여 벡터를 생성하면 x벡터는 숫자형으로 생성

- ✓ 논리형 TRUE, 문자형 "배 " 를 결합하여 벡터를 생성하면 y벡터는 문자형으로 생성

- ✓ 논리형 TRUE, 숫자 1, 문자형 "배 " 를 결합하여 벡터를 생성하면 z벡터는 문자형으로 생성

- R 기초

- 기본 사용법

- 벡터

- ✓ c 명령어를 이용하여 벡터와 벡터를 결합해 새로운 벡터를 형성할 수도 있음

- ✓ `x<-c(1,2,3,4)`

- ✓ `y<-c("사과", "배", "바나나")`

- ✓ `xy<-c(x,y)`

- ✓ xy 벡터는 x 벡터가 가지고 있던 스칼라 값은 그대로 가지고 있지만 결합하면 문자형 벡터로 전환되어 문자열 벡터로 인식하게 됨

• R 기초

- 기본 사용법

➤ 벡터

- ✓ rep 함수는 지정한 벡터를 반복 횟수만큼 반복하는 함수
- ✓ rep(x, times, ...), 첫 번째 인수를 두 번째 인수만큼 반복하는 숫자 벡터 생성

주요 파라미터	설명
x	벡터
times	벡터 전체의 반복 횟수

```
> rep(1, 4)  
[1] 1 1 1 1
```

```
> rep(1:3, 3)  
[1] 1 2 3 1 2 3 1 2 3
```

- R 기초

- 기본 사용법

- 벡터

- ✓ seq 함수: 첫 번째 인수부터 두 번째 인수까지 1씩 증가하는 수열의 숫자 벡터를 생성
 - ✓ 옵션: by=n(n씩 증가하는 수열생성), length=m(전체수열의 개수가 m개가 되도록 자동적으로 증가하는 수열 생성)

```
> seq(1, 4)
[1] 1 2 3 4
> seq(1, 10, by=2)
[1] 1 3 5 7 9
> seq(1, 9, length=5)
[1] 1 3 5 7 9
```

- R 기초

- 기본 사용법

- 벡터

- ✓ paste 함수는 입력받은 문자열들을 하나로 붙여주며 'sep' 옵션을 통해 붙이고자 하는 문자열들 사이에 구분자를 삽입시킬 수 있음

```
> number=c(1:6)
> text=c("a","b","c")
> paste(number,text,sep='to')
[1] "1toa" "2tob" "3toc" "4toa" "5tob" "6toc"
```

• R 기초

- 기본 사용법

➤ 벡터

✓ 벡터에 저장된 값의 위치를 **인덱스**로 사용하여 각 요소에 접근할 수 있음

✓ 벡터의 인덱싱

인덱싱	설명
벡터명[n]	• 벡터의 n번째 요소 반환
벡터명[-n]	• 벡터에서 n번째 요소를 제외한 모든 요소 반환
벡터명[조건문]	• 지정한 조건문을 만족하는 요소 반환
벡터명[a:b]	• 벡터의 a번째 요소부터 b번째 요소까지 요소 반환

```
> x <- c(2,3,4,5)
> x[2]
[1] 3
> x[-2]
[1] 2 4 5
> x[2:3]
[1] 3 4
```

- R 기초

- 기본 사용법

- 벡터

- ✓ 기초적인 수치 계산: 사칙연산이 수행되며 벡터와 벡터간 사칙연산 수행 시 연산되는 벡터의 길이가 같아야 함

```
> a=1:5
> b=6:10
> a
[1] 1 2 3 4 5
> b
[1] 6 7 8 9 10
> a+b
[1] 7 9 11 13 15
> a-b
[1] -5 -5 -5 -5 -5
> a*b
[1] 6 14 24 36 50
> a/b
[1] 0.1666667 0.2857143 0.3750000 0.4444444 0.5000000
```

• R 기초

- 기본 사용법

➤ 벡터

- ✓ substr 함수는 문자열의 일부를 추출하는 함수
- ✓ substr(x, start, stop)

주요 파라미터	설명
x	• 문자형 벡터
start	• 시작 위치
stop	• 종료 위치

```
> a<-c("hello","apple")  
> substr(a,1,3)  
[1] "hel" "app"
```


- R 기초

- 기본 사용법

- 리스트

- ✓ 리스트는 (키, 값) 형태로 데이터를 저장하는 R의 모든 객체를 담을 수 있는 데이터 구조
 - ✓ 리스트는 `list(key=value, key=value, ...)` 형태로 데이터를 나열해서 정의함
 - ✓ 리스트는 각 객체에 이름을 지정하여 저장할 수 있으며 객체의 길이가 달라도 저장이 됨

```
> list(name="kk", height=80)
$name
[1] "kk"

$height
[1] 80
```

• R 기초

- 기본 사용법

➤ 행렬

- ✓ 행렬은 행과 열을 갖는 $m \times n$ 형태의 직사각형에 데이터를 나열한 데이터 구조임
- ✓ matrix함수를 사용하며 `matrix(data, nrow, ncol, byrow, dimnames)`

주요 파라미터	설명
data	• 행렬에 저장할 데이터(벡터)
nrow	• 행의 개수(기본값은 1)
ncol	• 열의 개수(기본값은 1)
byrow	• 행렬의 데이터 입력순서(기본값은 FALSE) • TRUE: 행을 기준으로 입력, FALSE: 열을 기준으로 입력
dimnames	• 행과 열의 이름 리스트

- R 기초

- 기본 사용법

- 행렬

- ✓ 행렬은 행과 열을 갖는 $m \times n$ 형태의 직사각형에 데이터를 나열한 데이터 구조임
 - ✓ matrix함수를 사용하며 matrix(data, nrow, ncol, byrow, dimnames)

```
> matrix(c(1:9), nrow=3, dimnames=list(c("t1", "t2", "t3"),  
+ c("c1", "c2", "c3")))  
      c1 c2 c3  
t1     1  4  7  
t2     2  5  8  
t3     3  6  9
```

- R 기초

- 기본 사용법

- 행렬

- ✓ 행렬함수

함수	설명	함수	설명
dim(x)	x행렬의 차원 확인	nrow(x)	n행렬 행의 수 확인
ncol(x)	x행렬 열의 수 확인	x[m,n]	m행, n열의 원소 추출
x[m,]	x행렬 m행에 접근	x[,n]	x행렬의 n열에 접근
rownames(x)	x행렬의 행 이름 출력	colnames(x)	x행렬의 열 이름 출력

- R 기초

- 기본 사용법

- 행렬

- ✓ 행렬에 대해 * 연산을 수행하는 경우 **스칼라 곱**의 결과를 얻어낼 수 있음

```
> a=matrix(c(1,2,3,4,5,6), nrow=2)
> a
      [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
> 2*a
      [,1] [,2] [,3]
[1,]    2    6   10
[2,]    4    8   12
```

• R 기초

- 기본 사용법

➤ 데이터 프레임

- ✓ 데이터 프레임은 벡터들의 집합으로 행렬과 유사한 2차원 목록 데이터 구조로 숫자, 문자 등 다양한 형식의 데이터 저장 가능함
- ✓ 특히 행렬과 다르게 **각 열이 서로 다른 데이터 타입**을 가질 수 있어 데이터 크기가 커져도 사용자가 다루기 수월함
- ✓ data.frame 함수로 데이터 프레임을 생성함, data.frame(변수명1=벡터1, ...)

주요 파라미터	설명
변수명=벡터	변수명과 해당 열에 저장할 벡터 지정

- ✓ 원소의 접근은 **데이터프레임변수[행, 열]**과 **데이터프레임변수\$변수명**을 이용함

- R 기초

- 기본 사용법

- 데이터 프레임

- ✓ 각 행이 하나의 관측치를 의미하고 각 열이 하나의 변수라고 가정할 수 있음

```
> a = c(1, 2, 3, 4)
> b = c("apple", "banana", "grape", "tomato")
> c = c(FALSE, TRUE, FALSE, FALSE)
> abc = data.frame(a, b, c)
> abc
```

	a	b	c
1	1	apple	FALSE
2	2	banana	TRUE
3	3	grape	FALSE
4	4	tomato	FALSE

• R 기초

- 기본 사용법

➤ 배열

- ✓ 배열은 3차원 또는 n차원까지 확장된 형태의 다차원 데이터임
- ✓ array 함수는 배열을 생성하는 함수로 `array(data, dim, dimnames)`로 수행됨

주요 파라미터	설명
data	• 배열에 저장할 데이터 지정 • 데이터를 지정하지 않으면 NA 값으로 대체
dim	• 배열의 차원을 c()로 묶어서 지정
dimnames	• 배열의 차원 이름 지정 • 이름을 지정하지 않을 경우 기본값은 NULL

- R 기초

- 기본 사용법

- 배열

- ✓ 배열은 3차원 또는 n차원까지 확장된 형태의 다차원 데이터임
 - ✓ array 함수는 배열을 생성하는 함수로 array(data, dim, dimnames)로 수행됨

```
> a=c("1행", "2행")
> b=c("1열", "2열")
> d=c("1자원", "2자원")
> array(1:8, dim=c(2, 2, 2),
+ dimnames=list(a, b, d))
, , 1자원
      1열 2열
1행    1    3
2행    2    4

, , 2자원
      1열 2열
1행    5    7
2행    6    8
```

- R 기초

- 기본 사용법

- 데이터 결합

- ✓ R에서 데이터의 결합을 위해 **rbind함수**, **cbind함수**를 사용함
 - ✓ rbind함수는 벡터, 행렬, 데이터 프레임의 **행**을 서로 결합하여 새로운 행렬 또는 데이터 프레임을 생성하는 함수
 - ✓ cbind함수는 벡터, 행렬, 데이터 프레임의 **열**을 서로 결합하여 새로운 행렬 또는 데이터 프레임을 생성하는 함수
 - ✓ rbind(데이터1, 데이터2, ...)
 - ✓ cbind(데이터1, 데이터2, ...)

- R 기초

- 기본 사용법

- 데이터 결합

- ✓ 명령어 rbind와 cbind를 사용해 벡터를 서로 합쳐 행렬을 만들 수 있음

```
> mx = matrix(c(1,2,3,4,5,6), ncol=2)
> mx
      [,1] [,2]
[1,]    1    4
[2,]    2    5
[3,]    3    6
> r1 = c(5, 5)
> rbind(mx, r1)
      [,1] [,2]
      1    4
      2    5
      3    6
r1     5    5
```

```
> c1=c(10,10,10)
> cbind(mx, c1)
      c1
[1,]  1  4 10
[2,]  2  5 10
[3,]  3  6 10
```

- R 기초

- 기본 사용법

- 반복문

- ✓ 반복문은 대표적인 제어문 중 하나로 특정 부분의 코드가 반복적으로 수행되도록 함

- ✓ for 반복문과 while 반복문이 있음

```
> for (i in 1:3){  
+ print(i)  
+ }  
[1] 1  
[1] 2  
[1] 3
```

```
> i <- 0  
> while(i<5){  
+ print(i)  
+ i<-i+1  
+ }  
[1] 0  
[1] 1  
[1] 2  
[1] 3  
[1] 4
```

- R 기초

- 기본 사용법

- 조건문

- ✓ 조건문은 참과 거짓에 따라 특정 코드가 수행될지 혹은 수행되지 않을지를 결정하는 제어문임

```
> number <- 5
> if (number < 5){
+ print('number는 5보다 작다')
+ } else if(number > 5){
+ print('number는 5보다 크다')
+ } else {
+ print('number는 5와 같다')
+ }
[1] "number는 5와 같다"
```

• R 기초

- 기본 사용법

➤ 패키지

- ✓ 패키지는 함수, 데이터, 코드 등을 모아놓은 집합
- ✓ 패키지를 사용하기 위해 `install.packages` 함수로 패키지를 설치 후 `library` 함수로 패키지를 메모리에 불러와야 함
- ✓ 패키지 함수

함수	설명
<code>install.packages("패키지명")</code>	패키지를 설치하는 함수
<code>library(패키지명)</code>	패키지를 메모리에 불러오는 함수

- R 기초

- 기본 사용법

- 데이터 전처리 패키지

- ✓ 데이터 전처리 패키지는 **전처리 작업에 필요한 함수**들을 모아놓은 패키지임
 - ✓ plyr 패키지는 원본 데이터를 분석하기 쉬운 형태로 나눠서 다시 새로운 형태로 만들어 주는 패키지임
 - ✓ plyr 패키지의 함수들은 **데이터 분할(split), 원하는 방향으로 특정 함수 적용(apply), 그 결과를 재조합(combine)**하여 반환함
 - ✓ 여러 함수로 처리해야 할 데이터를 분할, 함수 적용, 재조합 함수를 통해 한 번에 처리할 수 있는 효율적인 패키지
 - ✓ plyr 패키지 함수는 ****ply** 형태이고 첫번째 글자는 입력데이터의 형태, 두번째 글자는 출력 데이터의 형태임

- R 기초

- 기본 사용법

- 데이터 전처리 패키지

✓ plyr 패키지 함수는 ****ply** 형태이고 첫번째 글자는 입력데이터의 형태, 두번째 글자는 출력 데이터의 형태임

출력 입력	dataframe	list	array
	dataframe	list	array
dataframe	ddply	dlply	daply
list	ldply	llply	laply
array	adply	alply	aaply

- R 기초

- 기본 사용법

- 데이터 전처리 패키지

- ✓ 데이터 전처리 패키지는 **전처리 작업에 필요한 함수**들을 모아놓은 패키지임
 - ✓ reshape2 패키지는 **melt와 cast 함수**를 사용함
 - ✓ melt함수는 데이터를 특정 변수를 기준으로 녹여서 나머지 변수에 대한 세분화된 데이터를 만드는 함수이고 cast함수는 melt함수에 의해 녹은 데이터의 모양을 다시 **재조합하는 함수**

- R 기초

- 기본 사용법

- 데이터 전처리 패키지

- ✓ 데이터 전처리 패키지는 **전처리 작업에 필요한 함수**들을 모아놓은 패키지임
 - ✓ data.table 패키지는 연산속도가 매우 빨라 크기가 큰 데이터를 처리하거나 탐색하는 데 효과적임
 - ✓ 데이터 프레임과 유사하지만 보다 빠른 grouping이 가능한 패키지임
 - ✓ 데이터 테이블은 데이터 프레임으로 변환해서 사용이 가능함
 - ✓ 데이터 테이블은 **데이터 프레임과 동일하게 취급되므로 데이터 프레임에 적용할 수 있는 함수**들을 사용할 수 있음

• R 기초

- 기본 사용법

➤ 데이터 테이블 생성

함수	설명
<code>data.table(tag=value, ...)</code>	- 데이터 테이블 생성 함수 <code>tag(변수명)=value(값)</code>의 형태로 데이터 생성

```
> install.packages("data.table")
'C:/Users/samsung/AppData/Local/R/win-library/4
(왜냐하면 'lib'가 지정되지 않았기 때문입니다)
--- 현재 세션에서 사용할 CRAN 미러를 선택해 주세요 ---
URL 'https://cran.yu.ac.kr/bin/windows/contrib/
Content type 'application/zip' length 2289586 b
downloaded 2.2 MB

패키지 'data.table'를 성공적으로 압축해제하였고 MD5 sums 이 확

다운로드된 바이너리 패키지들은 다음의 위치에 있습니다
      C:\Users\Public\Documents\ESTsoft\Creat
> library(data.table)
data.table 1.14.8 using 2 threads (see ?getDTth
경고메시지 (들) :
패키지 'data.table'는 R 버전 4.3.1에서 작성되었습니다
```

```
> t<-data.table(x=c(1:3),y=c("가", "나", "다"))
> t
   x y
1: 1 가
2: 2 나
3: 3 다
```

- R 기초

- 기본 사용법

- 데이터 변환

- ✓ 데이터 유형 변환 함수
 - ✓ `as.character(x)`: x를 문자형으로 변환함
 - ✓ `as.numeric(x)`: x를 숫자형으로 변환함
 - ✓ `as.double(x)`: x를 실수형으로 변환함
 - ✓ `as.logical(x)`: x를 논리형(TRUE 또는 FALSE)으로 변환함
 - ✓ `as.integer(x)`: x를 정수형으로 변환, x가 실수(예: $x=3.14 \rightarrow 3$)이면 소수점을 버림
 - ✓ `as.data.table(x)`: x를 데이터 테이블로 변환함

• R 기초

- 기본 사용법

➤ 데이터 변환

- ✓ 데이터 유형 변환 함수
- ✓ `as.Date(x, format)`: x를 format에 맞춰 인식한 후 연도-월-일 순으로 변환

포맷	설명	유효한 값
%d	일(Day) 숫자	01~31
%a	요일 약자	Mon ~ Sun
%A	요일	Monday ~ Sunday
%m	월(month) 숫자	01~12
%b	월(month) 약자	Jan ~ Dec
%B	월(Month)	January ~ December
%y	2자리 연도	00 ~ 99
%Y	4자리 연도	0000 ~ 9999

- R 기초

- 기본 사용법

- 데이터 변환

- ✓ 데이터 유형 변환 함수(예시)

```
> a<-0:2
> a
[1] 0 1 2
> b<-as.character(a)
> b
[1] "0" "1" "2"
> c<-as.logical(a)
> c
[1] FALSE TRUE TRUE
> as.Date("01/31/2030", "%m/%d/%Y")
[1] "2030-01-31"
```

• R 기초

- 기본 사용법

➤ 데이터 변환

✓ 자료구조 변환 함수

함수	설명
as.data.frame(x)	• x데이터를 데이터 프레임으로 변환
as.list(x)	• x데이터를 리스트로 변환
as.matrix(x)	• x데이터를 행렬로 변환
as.vector(x)	• x데이터를 벡터로 변환 • 데이터는 동일 자료형을 가져야 함

- R에서 서로 다른 데이터 타입을 담을 수 있는 구조는 무엇인가?
 - ① 행렬(Matrix)
 - ② 벡터(Vector)
 - ③ 리스트(List)
 - ④ 배열(Array)

- R에서 다음과 같이 matrix 함수를 이용하여 벡터를 행렬로 변환하였다고 할 때 생성된 mx의 결과는 무엇인가?

`mx = matrix(c(1,2,3,4,5,6), ncol=2, byrow=T)`

①

	[, 1]	[, 2]
[1,]	1	2
[2,]	3	4
[3,]	5	6

③

	[, 1]	[, 2]
[1,]	1	4
[2,]	2	5
[3,]	3	6

②

	[, 1]	[, 2]	[, 3]
[1,]	1	2	3
[2,]	4	5	6

④

	[, 1]	[, 2]	[, 3]
[1,]	1	3	5
[2,]	2	4	6

- 다음 중 08/23/2019을 "2019-08-23"으로 나타내는 코드로 올바른 것은?
 - ① `as.Date('08/23/2019', '%m/%d/%Y')`
 - ② `as.Date('08/23/2019', '%m/%D/%Y')`
 - ③ `as.Date('08/23/2019', '%M/%d/%Y')`
 - ④ `as.Date('08/23/2019', '%M/%D/%Y')`

- R의 데이터 구조 중 벡터에서 숫자형 벡터, 문자형 벡터, 논리 연산자 벡터를 모두 합쳐 하나의 벡터를 구성하였을 경우 합쳐진 벡터의 형식으로 옳은 것은?
 - ① 논리 연산자 벡터
 - ② 숫자형 벡터
 - ③ 문자형 벡터
 - ④ 데이터 프레임

- R의 ply패키지는 apply함수에 기반에 데이터와 출력변수를 동시에 배열로 치환하여 처리하는 패키지로 빠른 처리 속도를 보인다. 다음 중 입력되는 데이터 형태가 리스트이고 출력되는 데이터 형태가 데이터 프레임일 때 사용되는 함수는?

- ① ddply()
- ② adply()
- ③ ldply()
- ④ laply()

- 다음 중 코드의 결과로 적절한 것은?

```
s<-c("Monday", "Tuesday", "Wednesday")
```

```
substr(s, 1, 2)
```

① "Monday", "Tuesday"

② "Mo", "Tu", "We"

③ "Mo", "Tu"

④ "ay", "ay", "ay"

- 다음 프로그램을 통해 생성된 벡터 xy에 대한 설명으로 옳지 않은 것은?

```
x<-c(1:4)
```

```
y<-c("apple","banana","orange")
```

```
xy<-c(x,y)
```

- ① xy는 문자형 벡터이다.
- ② xy의 길이는 7이다.
- ③ xy[1]+xy[2]의 결과는 3이다.
- ④ xy[5:7]은 y와 동일하다.

- 다음은 R코드로 생성되는 행렬 A에서 일부 원소를 추출하기 위한 코드이다.

다음 중 나머지 보기와 결과가 다른 것은?

```
A<-cbind(c(1,2,3), c(4,5,6), c(7,8,9))
```

```
colnames(A) <- c("A", "B", "C")
```

```
rownames(A) <- c("a", "b", "c")
```

- ① A[, "A"]
- ② A[-c(2,3),]
- ③ A[, 1]
- ④ A[, -(2:3)]

- 다음 함수 중 원본 데이터를 분석하기 쉬운 형태로 나눠서 다시 새로운 형태로 만들어 주는 패키지로 가장 적절한 것은?

① plyr

② base

③ caret

④ ggplot2

- R패키지 설치 및 로드에 알맞은 것은?
 - ① `install.package("패키지명"), library("패키지명")`
 - ② `install.packages("패키지명"), library(패키지명)`
 - ③ `install.package(패키지명), library("패키지명")`
 - ④ `install.packages(패키지명), library(패키지명)`

- R 코드의 결과로서 옳지 않은 것은?

```
x <- 1:5
```

```
y <- seq(10, 50, 10)
```

```
t <- rbind(x,y)
```

① dim(t)의 결과는 [1] 2 5이다.

② t[1,]은 x와 같다.

③ t[, 1]은 y와 같다.

④ rbind()를 하기 위해서 결합하려는 두 개의 데이터 세트의 열의 개수가 같아야 한다.

- 벡터에 대한 설명으로 옳은 것은?

- ① R에서 벡터는 하나 이상의 스칼라 원소들을 갖는 집합이다.
- ② 문자형이 아닌 벡터를 합칠 때 문자형 벡터가 포함되면 합쳐지는 벡터는 문자형이 아닌 벡터형을 갖는다.
- ③ 논리형 벡터를 숫자형 벡터처럼 사용하는 경우 TRUE는 0이 된다.
- ④ 원소의 개수가 서로 다른 벡터끼리의 연산을 수행할 때는 원소의 개수가 적은 벡터의 개수만큼만 연산한다.

• $y=c(1,2,3,NA)$ 일 때, $3*y$ 의 결과는?

① NA, NA, NA, NA

② 3, 6, 9, NaN

③ 3, 6, 9, 3NA

④ 3, 6, 9, NA

- R의 패키지의 설명으로 부적절한 것은?

- ① data.table 패키지에서 리스트로 변경되면 ddply를 사용할 수 있다.
- ② data.table은 데이터 프레임과 유사하지만 보다 빠른 grouping이 가능하다.
- ③ ply() 함수는 앞에 두 개의 문자를 접두사로 가지는데 첫 번째 문자는 입력하는 데이터 형태를 나타내고 두번째 문자는 출력하는 데이터 형태를 나타낸다.
- ④ reshape2 패키지의 주요 기능은 melt()와 cast()가 있다.

- 다음 중 통계 패키지 R에 대한 설명으로 가장 부적절한 것은?
 - ① R은 오픈소스 프로그램이다.
 - ② 다양한 최신 통계 분석과 데이터마이닝 기능을 제공한다.
 - ③ Linux 환경에서 사용이 불가능하다.
 - ④ 사용자들이 여러 예시들을 공유한다.

- R 데이터의 저장 형식 설명으로 부적절한 것은?
 - ① `as.vector(matrix(c(1,2,3,4), nrow=2))`는 1 3 2 4 순으로 출력된다.
 - ② `as.integer(3.14)`의 값은 3이다.
 - ③ `as.numeric(FALSE)`의 값은 0이다.
 - ④ `as.logical(0.45)`의 값은 TRUE이다.

모듈 6-2

데이터 마트

- 데이터 마트

- 데이터 마트의 개념

- 데이터 마트는 데이터의 한 부분으로 특정 사용자가 관심을 두는 데이터들을 주제별, 부서별 및 목적별로 모아놓은 **비교적 작은 규모**의 데이터 웨어하우스
 - 잘 정리된 데이터마트의 개발은 효율적이고 신속한 모델링을 실시하는 경우 도움을 줌
 - ✓ 모델링은 다양한 분석기법을 적용해 모델을 개발하는 과정을 의미함

• 데이터 마트

- 데이터 저장소의 종류

구분	설명
데이터 웨어하우스	• 사용자의 의사결정 에 도움을 주기 위해 데이터베이스에 축적된 데이터를 공통 형식으로 변환해서 관리하는 데이터베이스
데이터 마트	• 전사적으로 구축된 데이터 속의 특정 주제, 부서 중심 으로 구축된 작은 규모의 데이터 웨어하우스
데이터 레이크	• 정형, 반정형, 비정형 데이터를 비롯한 모든 가공되지 않은 다양한 종류 의 데이터를 저장할 수 있는 시스템
데이터 댐	• 모든 산업의 데이터를 데이터 댐에 쌓는다는 의미로 어떤 값을 포함하고 있는 가공되지 않은 1차 자료를 모아놓은 저장소

• 다음 중 데이터의 한 부분으로 특정 사용자가 관심을 두는 데이터를 모아놓은 비교적 작은 규모의 데이터 웨어하우스를 무엇이라고 하는가?

- ① 데이터 베이스
- ② 데이터 마트
- ③ 데이터 마이닝
- ④ 데이터 프레임

- 데이터의 한 부분으로 특정 사용자가 관심이 있는 데이터를 담은 비교적 작은 규모의 데이터 웨어하우스는 무엇인가?

모듈 6-3

결측값 처리와 이상값 검색

- 결측값 처리와 이상값 검색

- 데이터 탐색

- 데이터의 특성을 파악하고 데이터에 대한 통찰을 얻기 위한 다양한 각도의 접근 방법
 - 데이터 탐색에 사용되는 함수는 head, str, summary 함수가 있음
 - ✓ head 함수: 데이터의 앞부분을 출력

주요 파라미터	설명
x	출력을 위한 객체
n	출력할 데이터 개수(기본값 6)

- 결측값 처리와 이상값 검색

- 데이터 탐색

- str 함수: 데이터의 속성을 출력

- ✓ str(object)

주요 파라미터	설명
object	데이터 속성을 출력하기 위한 객체

- 결측값 처리와 이상값 검색

- 데이터 탐색

- summary 함수

- ✓ 요약통계량을 출력하는 함수
 - ✓ 수치형 변수의 경우 최소값, 제1사분위수, 중위수, 평균, 제3사분위수, 최대값을 보여줌
 - ✓ 범주형 변수의 경우 각 범주에 대한 빈도수 출력
 - ✓ 변수 중 결측값이 있는 경우 결측값의 개수를 보여줌
 - ✓ summary(object)

주요 파라미터	설명
object	요약 통계량을 위한 객체

- 결측값 처리와 이상값 검색

- 데이터 탐색

- head 함수 예제

- ✓ mtcars 데이터 세트에 대해 앞에서 6개 데이터 출력

```
> head(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

- 결측값 처리와 이상값 검색

- 데이터 탐색

- head 함수 예제

- ✓ mtcars 데이터 세트의 앞에서 10개 행 출력

```
> head(mtcars, 10)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4

- 결측값 처리와 이상값 검색

- 데이터 탐색

- str 함수 예제

- ✓ mtcars 데이터 세트의 구조 파악(11개 변수, 32개의 행으로 구성)

```
> str(mtcars)
'data.frame':   32 obs. of  11 variables:
 $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
 $ cyl : num   6  6  4  6  8  6  8  4  4  6 ...
 $ disp: num  160 160 108 258 360 ...
 $ hp  : num  110 110  93 110 175 105 245  62  95 123 ...
 $ drat: num   3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
 $ wt  : num   2.62 2.88 2.32 3.21 3.44 ...
 $ qsec: num  16.5 17 18.6 19.4 17 ...
 $ vs  : num   0  0  1  1  0  1  0  1  1  1 ...
 $ am  : num   1  1  1  0  0  0  0  0  0  0 ...
 $ gear: num   4  4  4  3  3  3  3  4  4  4 ...
 $ carb: num   4  4  1  1  2  1  4  2  2  4 ...
```

- 결측값 처리와 이상값 검색

- 데이터 탐색

- summary 함수 예제

- ✓ mtcars 데이터 세트의 기초 통계량 확인

```
> summary(mtcars)
```

mpg	cyl	disp	hp
Min. :10.40	Min. :4.000	Min. : 71.1	Min. : 52.0
1st Qu.:15.43	1st Qu.:4.000	1st Qu.:120.8	1st Qu.: 96.5
Median :19.20	Median :6.000	Median :196.3	Median :123.0
Mean :20.09	Mean :6.188	Mean :230.7	Mean :146.7
3rd Qu.:22.80	3rd Qu.:8.000	3rd Qu.:326.0	3rd Qu.:180.0
Max. :33.90	Max. :8.000	Max. :472.0	Max. :335.0

drat	wt	qsec	vs
Min. :2.760	Min. :1.513	Min. :14.50	Min. :0.0000
1st Qu.:3.080	1st Qu.:2.581	1st Qu.:16.89	1st Qu.:0.0000
Median :3.695	Median :3.325	Median :17.71	Median :0.0000
Mean :3.597	Mean :3.217	Mean :17.85	Mean :0.4375
3rd Qu.:3.920	3rd Qu.:3.610	3rd Qu.:18.90	3rd Qu.:1.0000
Max. :4.930	Max. :5.424	Max. :22.90	Max. :1.0000

am	gear	carb
Min. :0.0000	Min. :3.000	Min. :1.000
1st Qu.:0.0000	1st Qu.:3.000	1st Qu.:2.000
Median :0.0000	Median :4.000	Median :2.000
Mean :0.4062	Mean :3.688	Mean :2.812
3rd Qu.:1.0000	3rd Qu.:4.000	3rd Qu.:4.000
Max. :1.0000	Max. :5.000	Max. :8.000

- 결측값 처리와 이상값 검색

- 결측값 처리

- 결측값의 개념

- ✓ 결측값은 입력이 누락된 값으로 가능하면 제외하고 처리하는 것이 적합함
 - ✓ 그러나 결측값 자체가 의미가 있는 경우도 가끔 존재할 수도 있음
 - ✓ 결측값에 대한 처리가 전체 작업속도에 많은 영향을 주므로 자동화를 하면 효율성이 향상됨

- 결측값 처리와 이상값 검색

- 결측값 처리

- 결측값의 확인

- ✓ 결측값을 입력하는 방법으로 NA를 이용함
 - ✓ 결측값은 `is.na(x)`, `complete.cases(x)` 함수를 이용하여 결측값 확인 및 삭제를 수행함

함수	설명
<code>is.na(x)</code>	• 데이터의 각 행과 변수별로 결측값이 있으면 TRUE, 아니면 FALSE를 출력
<code>complete.cases(x)</code>	• 데이터의 행별로 결측값이 없으면 TRUE, 있으면 FALSE 출력

- 결측값 처리와 이상값 검색

- 결측값 처리

- 결측값 예제

- ✓ is.na함수 사용

```
> data=c(1,2,3,4,NA)
> is.na(data)
[1] FALSE FALSE FALSE FALSE TRUE
```

- 결측값 처리와 이상값 검색

- 결측값 처리

- 결측값 예제

- ✓ is.na, complete.cases 함수 사용

```
> na_sample=head(airquality, 5)
> na_sample
  Ozone Solar.R Wind Temp Month Day
1   41     190  7.4   67     5    1
2   36     118  8.0   72     5    2
3   12     149 12.6   74     5    3
4   18     313 11.5   62     5    4
5   NA       NA 14.3   56     5    5
```

```
> complete.cases(na_sample)
[1] TRUE TRUE TRUE TRUE FALSE
```

```
> is.na(na_sample)
  Ozone Solar.R Wind Temp Month Day
1 FALSE   FALSE FALSE FALSE FALSE
2 FALSE   FALSE FALSE FALSE FALSE
3 FALSE   FALSE FALSE FALSE FALSE
4 FALSE   FALSE FALSE FALSE FALSE
5  TRUE    TRUE  FALSE FALSE FALSE
```


- 결측값 처리와 이상값 검색

- 결측값 처리

- 결측값 처리 방법

- ✓ 단순 대체법: 결측값을 그럴듯한 값으로 대체하는 통계적 기법으로
 - ✓ 통계량의 효율성 및 일치성 등의 문제를 부분적으로 보완해주어
 - ✓ 대체된 자료는 결측값이 없이 완전한 형태를 지니게 됨
 - ✓ 단순 대체법에는 **완전 분석법**, **평균 대체법**, **단순확률 대체법**이 있음

• 결측값 처리와 이상값 검색

- 결측값 처리

➤ 결측값 처리 방법

✓ 단순대치법

종류	설명
완전 분석법 (단순 삭제)	- 불완전한 자료는 모두 무시하고 완전하게 관측된 자료만 사용하여 분석 - 분석은 쉽지만 데이터 활용 변수가 작아져 효율성이 상실되고 통계적 추론의 타당성이 부족함
평균 대치법	- 관측이나 실험으로 얻어진 자료의 평균값으로 결측값을 대치하는 방법 - 비 조건부 평균 대치법: 자료의 평균값 사용 - 조건부 평균 대치법: 결측값이 있는 변수를 종속변수로 하여 회귀분석을 실시
단순확률 대치법	- 평균대치법에서 대치값을 어떤 적절한 확률값을 부여한 후 대치하는 방법 - 표준오차(평균의 표준편차)에 대한 과소추정 문제를 보완하고자 고안하였지만 간단한 경우를 제외하고 추정량의 표준오차 계산 자체가 어려움

• 결측값 처리와 이상값 검색

- 결측값 처리

➤ 결측값 처리 방법

- ✓ 다중 대치법: 단순 대치법을 m번 수행하여 m개의 가상적 완전한 자료를 만들어서 분석하는 방법으로
- ✓ 단순 대치법의 추정량 표준오차의 과소추정 또는 계산의 난해성 문제를 보완하는 방법으로
- ✓ 대치단계 → 분석단계 → 결합단계의 3단계로 구성되어 있음

단계	설명
대치	• 다양한 대치 방법을 이용하여 가상의 완전한 자료 m개 데이터세트 생성
분석	• m개 데이터세트 각각에 대한 통계분석(모수추정치, 표준오차 등)을 실시
결합	• 분석 단계에서 생성된 m개의 추정량과 분산의 결합을 통한 통계적 추론

- 결측값 처리와 이상값 검색

- 이상값 검색

- 이상값 개념

- ✓ 이상값은 관측된 데이터의 범위에서 많이 벗어난 아주 작은 값이나 큰 값을 의미함
 - ✓ 입력의 오류 또는 데이터처리 오류 등의 이유로 특정 범위에서 벗어난 데이터 값

- 이상값 검색방법

- ✓ 이상값 검출 방법으로 평균과 표준편차를 이용한 ESD(Extreme Studentized Deviation)방법과 사분위수를 이용한 방법을 많이 이용함
 - ✓ 데이터 분석에서 전처리 방법을 결정할 때 부정 사용 방지 시스템(FDS: Fraud Detection System)의 규칙발견에 사용할 수 있음

• 결측값 처리와 이상값 검색

- 이상값 검색

➤ 이상값 검색방법

- ✓ ESD는 평균으로부터 표준편차(σ)의 k 배 떨어진 값을 이상값으로 판별하는 방법으로 $k=3$ 으로 하는 것이 일반적임($\text{표본평균} - 3 \times \sigma < \text{데이터} < \text{표본평균} + 3 \times \sigma$ (정상범주))
- ✓ ESD는 평균을 이용하기 때문에 이상값에 민감하다는 단점이 존재함
- ✓ 기하평균을 이용하는 방법은 기하평균으로부터 표준편차의 2.5배보다 떨어져 있는 값을 이상값으로 판단함
- ✓ 기하평균 - $2.5 \times \sigma < \text{데이터} < \text{기하평균} + 2.5 \times \sigma$ (정상범주)
- ✓ 기하평균: n 개의 양수 값을 모두 곱한 것의 n 제곱근
- ✓ 2, 3, 6의 기하평균 $\rightarrow (2 \times 3 \times 6)^{\frac{1}{3}} = 3.3$

- 결측값 처리와 이상값 검색

- 이상값 검색

- 이상값 검색방법

- ✓ 사분위수를 이용한 방법으로 제1사분위수(Q_1), 제3사분위수(Q_3)를 기준으로 사분위 범위($Q_3 - Q_1$)의 1.5배 떨어져 있는 값을 이상값으로 판단
 - ✓ $Q_1 - 1.5 \times (Q_3 - Q_1) < data < Q_3 + 1.5 \times (Q_3 - Q_1)$ (정상범주)
 - ✓ 사분위수를 이용하므로 ESD와 달리 이상값에 민감하지 않음

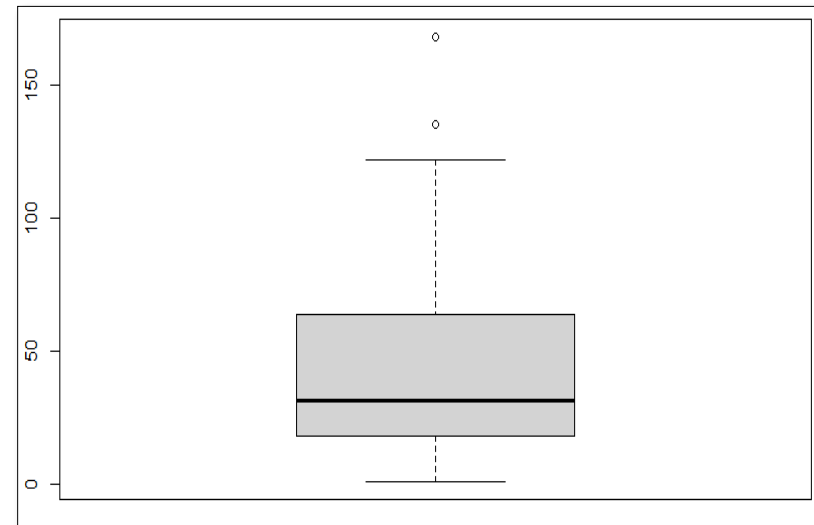
- 결측값 처리와 이상값 검색

- 이상값 검색

- 이상값 검색방법

- ✓ 박스플롯(상자그림)을 이용한 이상값 탐색

```
> #airquality 데이터 세트의 ozone 변수 박스플롯 그리기  
> out_oz = boxplot(airquality$ozone)  
> #이상값 출력  
> out_oz$out  
[1] 135 168
```



- 다음의 결측값 처리에 대한 내용 중 가장 옳지 않은 것은?

- ① 단순확률대치법은 추정량의 과소추정이나 계산의 난해성 문제를 보완하는 방법이다.
- ② 다중대치법은 단순 대치법을 한 번 하지 않고 m 번 대치를 통해 m 개의 가상적 완전한 자료를 만들어서 분석하는 방법이다.
- ③ 평균대치법은 관측 또는 실험되어 얻어진 자료의 평균값으로 결측값을 대치하는 방법이다.
- ④ 완전분석법은 불완전 자료는 모두 무시하고 완전하게 관측된 자료만 사용하여 분석하는 방법이다.

- 다음 중 이상값 검색 방법으로 가장 옳지 않은 것은?

- ① 3-Sigma방법은 평균으로부터 표준편차(σ)의 3배보다 떨어진 값을 이상값으로 판별하는 방법이다.
- ② Q_2 (중위수)로부터 IQR(사분위범위)의 1.5배보다 크거나 작은 데이터를 이상값으로 규정한다.
- ③ ESD는 이상값에 민감하다는 단점이 있다.
- ④ 기하평균으로부터 표준편차의 2.5배보다 떨어져 있는 값을 이상값이라 판단한다.

- 이상값 검색 기법의 하나로 평균으로부터 표준편차의 k 배보다 떨어진 값들을 이상값으로 판단하는 방법은 무엇인가?

- 다음 중 박스 플롯(Boxplot)을 이용한 이상값 검색에 대한 설명으로 가장 옳지 않은 것은 무엇인가?
 - ① 평균에서 표준편차의 3배보다 떨어진 값을 이상값으로 판단한다.
 - ② 이상값을 반드시 제거해야 하는 것은 아니므로 이상값을 처리할지는 분석의 목적에 따라 적절한 판단이 필요하다.
 - ③ 중위수를 이용하므로 이상값에 민감하지 않다.
 - ④ $Q_1 - 1.5 \times IQR < data < Q_3 + 1.5 \times IQR$ 을 벗어나는 data를 이상값이라고 판단한다.(단, $IQR = Q_3 - Q_1$ 이라고 한다.)

• 다음 중에서 이상값 검색을 활용한 응용시스템으로 가장 적절한 것은 무엇인가?

- ① 부정 사용 방지 시스템
- ② 장바구니 분석 시스템
- ③ 분류 분석 시스템
- ④ 군집 분석 시스템

- 다음 중 이상값을 판정하는 방법으로 가장 적절하지 않은 것은 무엇인가?
 - ① $Q_2 \pm 1.5 * IQR$ 보다 크거나 작은 경우 이상값으로 판정한다.
 - ② 기하평균으로부터 표준편차의 2.5배가 넘는 범위의 데이터를 이상값으로 판정한다.
 - ③ 회귀분석 후 잔차 분석을 실시하여 이상값을 판정한다.
 - ④ 평균으로부터 표준편차의 3배가 넘는 범위의 데이터를 이상값으로 판정한다.

- 최소값, 1사분위수, 중위수, 3사분위수, 최대값, 평균값을 구할 수 있는 함수는?

- ① str
- ② summary
- ③ head
- ④ inform

- summary(cars)에 대한 설명 중 적절하지 않은 것은?

```
> summary(cars)
      speed      dist
Min.   : 4.0    Min.   :  2.00
1st Qu.:12.0    1st Qu.: 26.00
Median :15.0    Median : 36.00
Mean   :15.4    Mean   : 42.98
3rd Qu.:19.0    3rd Qu.: 56.00
Max.   :25.0    Max.   :120.00
```

- ① dist변수의 최소값과 최대값을 알 수 있다.
- ② 두 변수의 중위수를 알 수 있다.
- ③ speed 변수의 제2사분위는 알 수가 없다.
- ④ dist변수의 평균값을 알 수 있다.

- summary(mtcars)에 대한 설명 중 적절하지 않은 것은?

```
> summary(mtcars)
```

mpg	cyl	disp	hp
Min. :10.40	Min. :4.000	Min. : 71.1	Min. : 52.0
1st Qu.:15.43	1st Qu.:4.000	1st Qu.:120.8	1st Qu.: 96.5
Median :19.20	Median :6.000	Median :196.3	Median :123.0
Mean :20.09	Mean :6.188	Mean :230.7	Mean :146.7
3rd Qu.:22.80	3rd Qu.:8.000	3rd Qu.:326.0	3rd Qu.:180.0
Max. :33.90	Max. :8.000	Max. :472.0	Max. :335.0

- ① cyl변수의 중위수는 6.00이다.
- ② disp변수를 벡터로 추출하기 위해서 mtcars\$disp를 실행한다.
- ③ hp변수는 범주형 데이터 타입이다.
- ④ mpg 변수의 최대값은 33.90이다.